



Práctica Inteligencia Artificial: Búsqueda Heurística en Radares

Inteligencia Artificial

Grado en Ingeniería Informática

Doble Grado en Ingeniería Informática y Administración de Empresas

Curso académico 2024-2025

Índice

1	Enunciado	2
2	Tareas	6
2.1	Modelado del problema de búsqueda	6
2.2	Generación del mapa y el espacio de búsqueda	6
2.3	Integración del algoritmo de búsqueda	6
2.4	Experimentación con distintos escenarios	6
3	Código proporcionado	7
4	Evaluación	8
4.1	Modelado del problema	8
4.2	Implementación del mapa	8
4.3	Implementación del sistema de búsqueda	8
4.4	Experimentación	8
4.5	Memoria	9
5	Normas	9
6	Uso de IA generativa	9

1. Enunciado

En el campo de la navegación aérea y la teledetección, el radar (*Radio Detection and Ranging*) constituye un sistema fundamental basado en la emisión y recepción de ondas electromagnéticas que permite detectar, localizar y rastrear (*tracking*) objetos móviles a una determinada distancia (véase la figura 1). Su funcionamiento se centra en la emisión de pulsos de radiofrecuencia que, al impactar con un objeto, son reflejados y regresan del vuelta al radar, permitiendo estimar la posición, velocidad y dirección del movimiento asociado a un objeto en particular.

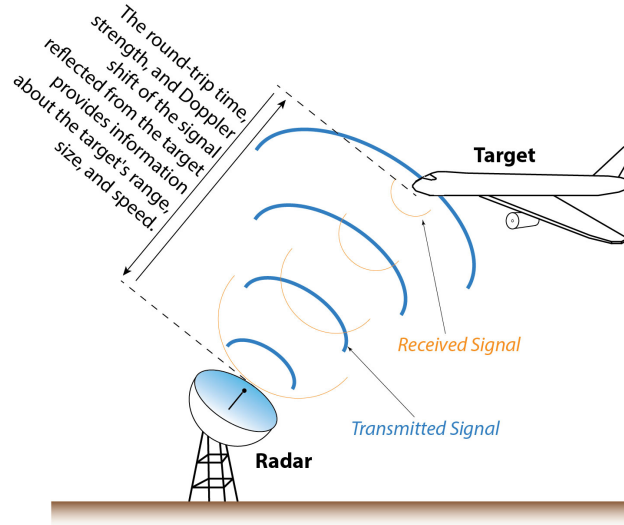


Figura 1: Funcionamiento de un radar.

Los radares poseen diversas propiedades que los caracterizan, como lo son:

- Alcance y precisión
- Capacidad de operar en condiciones adversas
- Diferenciación de objetos
- Estimación de la velocidad

De entre las cuales el alcance y la precisión es quizás una de las propiedades fundamentales más relevantes del radar. Por ello, ingenieros de varias ramas desarrollaron lo que hoy se conoce como ecuación del radar, un instrumento matemático que permite conocer la distancia máxima de detección de un radar concreto, utilizando como base sus propiedades físicas y operacionales. Matemáticamente:

$$R_{\text{máx}} = \left(\frac{P_t G^2 \lambda^2 \sigma}{(4\pi)^3 P_{\text{mín}} L} \right)^{\frac{1}{4}} \quad (1)$$

Donde:

- $R_{\text{máx}}$: Radio máximo de detección (distancia máxima) [m].
- P_t : Potencia transmitida [W].
- G : Ganancia de la antena (adimensional).
- λ : Longitud de onda de la señal [m].
- σ : Sección transversal de la antena (reflectividad del blanco) [m²].
- $P_{\text{mín}}$: Potencia mínima detectable por el receptor [W].
- L : Pérdidas adicionales por clima e impedancias (adimensional: $L \in \mathbb{Z}^+$)

Con la ecuación del radar (1), podemos conocer una muy buena aproximación de a qué distancia el radar dejará de detectar objetos de forma eficaz, por lo que, implícitamente, nos está proporcionando información sobre el rango de actuación del mismo.

En el problema propuesto, un avión espía sigiloso (aviones furtivos) está siendo testeado por el ejército. Su diseño emplea lo que se conoce como *tecnologías furtivas* que principalmente le permiten absorber y desviar las radiaciones emitidas por el radar, evitando el “rebote” de los pulsos de radiofrecuencia que permiten al radar identificar objetos móviles. Sin embargo, no es completamente invisible. La presencia de varios radares y la avanzada tecnología de los mismos hace que el avión no pueda sobrevolar cualquier espacio sin el riesgo de ser detectado. El objetivo del avión espía consiste en sobrevolar y fotografiar desde el aire distintas regiones críticas sin ser detectado. Para lograr esto, el ejército quiere desarrollar una herramienta informática que le permita al avión disponer de un mapa de probabilidades de ser detectado, para que, mediante el uso de algoritmos de búsqueda heurística, el avión pueda realizar un plan que le permita visitar las zonas críticas minimizando la exposición total a los radares.

Para construir este mapa de detección, el ejército ha definido las siguientes directrices de diseño:

1. El mapa será una región en forma de malla (rejilla) discreta que cuente con $H \times W$ bloques, siendo H el número de bloques que posee cada columna de la malla y siendo W el número de bloques que posee cada fila de la malla.
2. La malla cubrirá un área real, delimitada por unas fronteras expresadas en coordenadas geodésicas, es decir, el área que cubre la malla tendrá una forma rectangular y ésta tendrá unas coordenadas (lat_0, lon_0) para la esquina superior izquierda del área y otras coordenadas (lat_1, lon_1) para la esquina inferior derecha del área. Veamos el siguiente ejemplo: tenemos un área delimitada por $\{(lat_0, lon_0), (lat_1, lon_1)\} = \{(40.10^\circ, -3.18^\circ), (40.03^\circ, -3.07^\circ)\}$, donde se desea construir una malla de $H = 20$ y $W = 30$, por lo que el recorrido vertical en latitud $\Delta_{lat} = lat_0 - lat_1 = 40.10^\circ - 40.03^\circ = 0.07^\circ$ será repartido en $H = 20$ puntos, de manera que el primer y último punto serán $h_0 = 40.03^\circ$ y $h_{19} = 40.10^\circ$ respectivamente. De igual forma el recorrido horizontal en longitud $\Delta_{lon} = lon_1 - lon_0 = -3.07^\circ - (-3.18^\circ) = 0.11^\circ$ que serán repartidos en $W = 30$ puntos, de tal forma que el primer y último punto serán $w_0 = -3.18^\circ$ y $w_{29} = -3.07^\circ$, respectivamente. Para más información, véase la figura 2.
3. Cada radar tomará unas coordenadas $R_i = (lat_i, lon_i)$ que se encontrarán contenidas en el área delimitadora, expuesta en el punto anterior.
4. Cada punto (bloque o celda) de la malla codificará la probabilidad de ser detectado por cualquiera de los radares. La posibilidad (ojo, no la probabilidad aún) de ser detectado por un radar en particular, se modela mediante una gaussiana multivariante, cuya formulación matemática es la que sigue a continuación:

$$\Psi(x_1, x_2, \dots, x_n) = \frac{1}{(2\pi)^{n/2} |\Sigma|^{1/2}} e^{-\frac{1}{2}(x-\mu)^T \Sigma^{-1} (x-\mu)} \quad (2)$$

Donde $\Psi(x_1, x_2, \dots, x_n)$ representa la posibilidad (o verosimilitud, no la probabilidad), Σ representa la matriz de covarianzas y donde μ representa el vector de medias $\mu = \{\mu_{x_1}, \mu_{x_2}, \dots, \mu_{x_n}\}$. En nuestro problema, la media μ serán las coordenadas del radar y la matriz de covarianzas Σ será generada automáticamente. Esto quiere decir que nuestra gaussiana multivariante será bidimensional, lo cual simplifica $\Psi(x_1, x_2, \dots, x_n)$ a $\Psi(x_1, x_2) = \Psi(lat, lon)$. Como puede haber más de un radar, para calcular la posibilidad de detección en un punto en concreto, emplearemos la siguiente expresión:

$$\Psi^*(lat, lon) = \max_{\Psi_i^*} \{\Psi_i^*(lat, lon)\}_{i=1}^{N_r} \quad (3)$$

Donde:

$$\Psi_i^*(lat, lon) = \begin{cases} \Psi_i(lat, lon) & \text{si } d \leq R_{\text{máx}} \\ 0 & \text{si } d > R_{\text{máx}} \end{cases} \quad (4)$$

De manera que $\Psi^*(lat, lon)$ representa la máxima posibilidad de ser detectado para cada uno de los i -radares existentes. Esto puede ser calculado mediante el cómputo de la posibilidad de cada uno de los i -radares

(denotado por Ψ_i^* , donde el número de radares viene representado por N_r) de manera que se almacene únicamente el valor Ψ_i^* que maximiza la posibilidad de detección. Por último se recuerda que Ψ_i^* tomará el valor 0 si la distancia entre el punto sobre el que se quiere conocer la posibilidad de detección y el radar que maximiza la posibilidad es menor que la distancia $R_{\text{máx}}$ determinada por la ecuación del radar (1). Para calcular la distancia d entre el punto y el radar, se empleará la distancia euclídea en dos dimensiones:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2} \quad (5)$$

Es importante recordar que la distancia euclídea se está calculando entre coordenadas geodésicas, de manera que las unidades serán grados decimales, sin embargo, la ecuación del radar emplea los metros como unidad de distancia, por lo que una aproximación a la distancia entre dos puntos dadas sus coordenadas geodésicas se calcularía como:

$$d = K \cdot \sqrt{(lat_{point} - lat_{radar})^2 + (lon_{point} - lon_{radar})^2} \quad (6)$$

Donde $K = 111000$ metros, que es una aproximación decente para convertir un grado geodésico en metros. Esta aproximación funciona mejor para los grados de la latitud que los de la longitud, ya que la longitud se desglosa en meridianos que se aproximan entre sí conforme se acercan a los polos, por lo que la aproximación que aporta K no es constante. Para el problema que trataremos, bastará con aplicar la fórmula 6 para estimar la distancia en metros.

5. Para el cálculo de la probabilidad a partir de la posibilidad, se indica que cada valor $\Psi^* \in [0, +\infty]$ sea escalado mediante la técnica *MinMax*, que opera como sigue:

$$\Psi_{scaled}^* = \frac{\Psi^* - \Psi_{\text{mín}}^*}{\Psi_{\text{máx}}^* - \Psi_{\text{mín}}^*} \quad (7)$$

Logrando que $\Psi_{scaled}^* \in [0, 1]$. Al haber escalado de esta forma, existirán casillas que poseen un valor de detección igual a cero. Esto dificulta el diseño de heurísticas admisibles, ya que podrían existir caminos con coste total igual a cero, lo cual aumenta considerablemente la dificultad requerida a la hora de estimar costes que no sobreestimen al real. Para evitar que haya casillas con probabilidad igual a cero, se establece un valor ϵ muy cercano a cero, que permitirá agregar un coste residual a cualquier casilla que tenga coste nulo. Partiendo de la ecuación 7, se realiza la siguiente modificación:

$$\Psi_{scaled}^* = \frac{\Psi^* - \Psi_{\text{mín}}^*}{\Psi_{\text{máx}}^* - \Psi_{\text{mín}}^*} \cdot (1 - \epsilon) + \epsilon \quad (8)$$

De esta forma, logramos que $\Psi_{scaled}^* \in [\epsilon, 1]$, evitando casillas con valores nulos y por tanto evitando transiciones en el espacio de búsqueda sin un coste asociado.

6. El avión espía no podrá desplazarse por cualquier casilla de la malla si ésta supera un determinado umbral / tolerancia que será aportado por los argumentos del programa. Si el umbral toma el valor 0.5, significará que el avión no podrá transicionar hacia casillas que poseen valores de probabilidad superiores. Con esto se pretende modelar el riesgo que toma el avión espía al pasar más o menos cerca del campo de detección de los radares.
7. El avión espía debe visitar una secuencia de POI's (*Points Of Interest*), de manera que inicialmente se deberá establecer un "plan de alto nivel" donde se decide en qué orden visitar los puntos, para luego emplear el algoritmo A^* y la búsqueda heurística como herramienta algorítmica para encontrar el camino mínimo entre cada par de puntos. Para entender esto, se aporta el siguiente ejemplo: $POI = \{p_1, p_2, p_3, p_4, p_5\}$, se calcula que el orden de visita de los puntos sea $\{p_1, p_3, p_5, p_2, p_4\}$, de manera que para construir el plan que realizará el avión espía se tendrá que emplear un $A^*(p_1, p_3)$, $A^*(p_3, p_5)$, $A^*(p_5, p_2)$ y $A^*(p_2, p_4)$. La concatenación de cada uno de estos planes será la solución al problema (véase la figura 3).

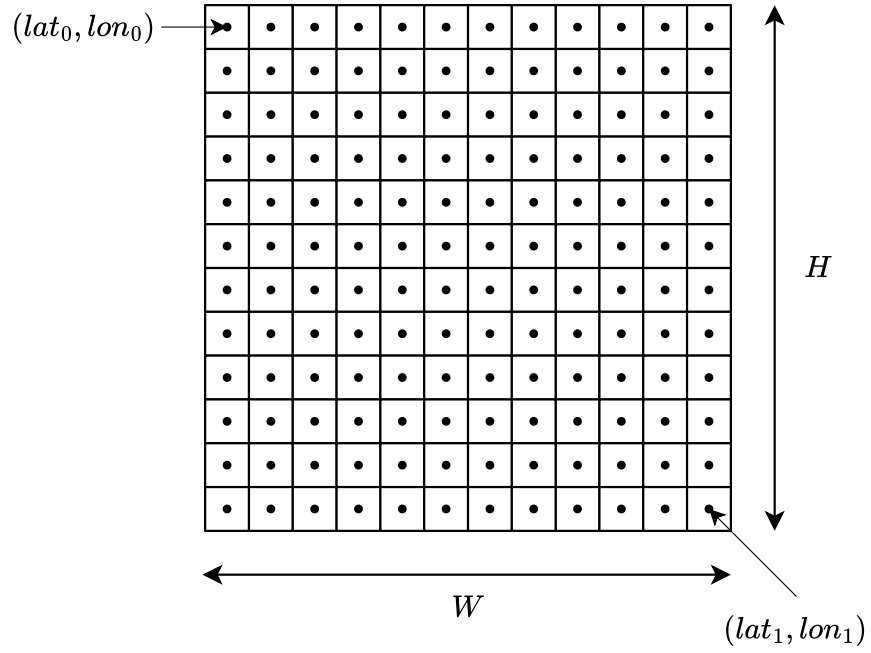


Figura 2: Ejemplo de malla recubridora de un área delimitada por (lat_0, lon_0) y (lat_1, lon_1) , con dimensiones $H \times W$.

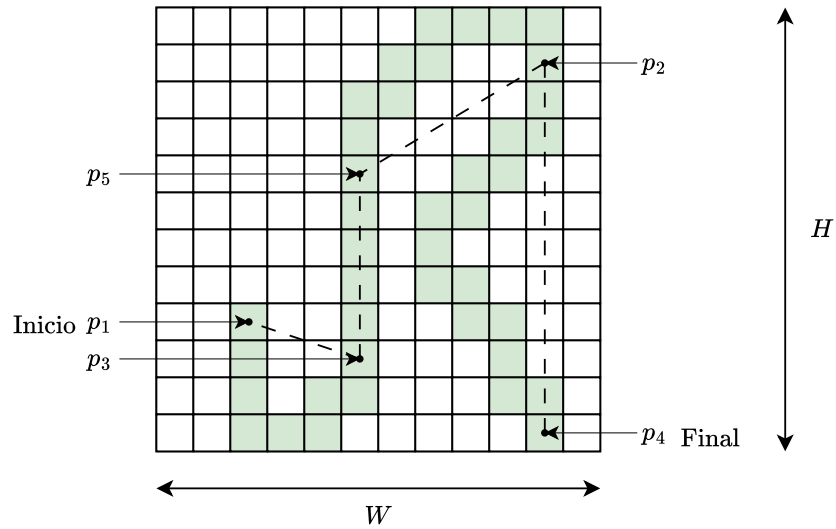


Figura 3: Ejemplo de plan de alto nivel y planificación local empleando A^* . Las casillas en color verde representan la selección de pasos que debe realizar el avión. Es importante recordar que cada casilla en la malla posee una probabilidad de detección determinada, por ello, el camino verde toma la ruta que minimiza la detección.

2. Tareas

Para lograr los objetivos propuestos, la práctica queda desglosada en la siguiente secuencia de tareas que deben ser completadas a modo de ejercicios separados pero relacionados entre sí.

2.1. Modelado del problema de búsqueda

Para este apartado se pide:

1. Modelar matemáticamente los estados y operadores del problema. Esto se realizará en la memoria de la práctica y deberá cubrir todos los aspectos relevantes para resolver el problema, empleando rigor y formalidad matemática. En los modelos y definiciones formales, no debe existir la ambigüedad o el exceso de texto sin aportaciones. Emplear símbolos, ecuaciones, conjuntos, etc, permite obtener un modelo claro y conciso del problema.
2. Definir el estado inicial y las características del estado final del problema de búsqueda, empleando el modelo de estados que se realizó en el apartado anterior.
3. Diseñar y formular como mínimo dos heurísticas admisibles para el problema de búsqueda. Es muy importante que el alumno se asegure de que las heurísticas sean admisibles, es decir, que el coste estimado por las heurísticas nunca sobrepase el coste real del problema. Para garantizar la admisibilidad no existe un procedimiento estándar, por ello, el alumno debe demostrar / justificar la admisibilidad de sus heurísticas. El diseño de heurísticas no admisibles que funcionen para el problema serán admitidas pero no obtendrán toda la calificación disponible.

2.2. Generación del mapa y el espacio de búsqueda

Para este apartado se pide:

1. Implementar la generación del mapa (malla) sobre el que se define el espacio de búsqueda. Para ello, se tendrá que programar el cálculo de la probabilidad de detección en cada casilla, empleando la ecuación del radar (1), la ecuación de posibilidad de detección (3) y la ecuación de escalado (8), que permitirán construir un mapa almacenado en memoria con cada uno de los valores de detección por celda.
2. Implementar la generación de un grafo que defina el espacio de búsqueda a partir del mapa de detecciones. Como la implementación del algoritmo A^* trabaja directamente sobre grafos, los alumnos deberán implementar la construcción de un grafo a partir de un mapa de detecciones. Debido a que los desplazamientos posibles que puede realizar el avión espía son hacia las casillas colindantes (arriba, abajo, izquierda y derecha), lo que se debe conseguir es generar tantos vértices como casillas tenga la malla, y tantas aristas como posibles conexiones haya entre vecinos colindantes para cada casilla en particular. Como durante la generación del mapa las casillas son las que poseen un valor de probabilidad de detección, las aristas que unirán los vértices del grafo tomarán como pesos (costes) el valor del vértice (casilla) hacia el que se realiza la transición. Es decir, si el vértice v_i representa a una casilla con probabilidad de detección igual a 0.88 y el vértice v_j representa a una casilla con probabilidad de detección 0.27, la arista que une i con j tendrá un peso de $e_{ij} = 0.27$ y la que une j con i tendrá un peso asociado de $e_{ji} = 0.88$. Para más información, véase la figura 4.

2.3. Integración del algoritmo de búsqueda

Empleando la implementación del algoritmo A^* de la biblioteca `networkx` de Python, denominada `astar_path`, el alumno deberá ser capaz de integrar el algoritmo en el sistema de búsqueda, para resolver el problema de la ruta (*pathfinding*). Es importante recordar que las heurísticas que se diseñaron en el apartado 2.1. deben estar implementadas para poder ser aplicadas por el algoritmo A^* .

2.4. Experimentación con distintos escenarios

Junto con el enunciado se proporcionarán distintos escenarios que plantean mallas de diferentes dimensiones con conjuntos de radares distintos distribuidos a lo largo de la superficie. En estos escenarios se aporta también la

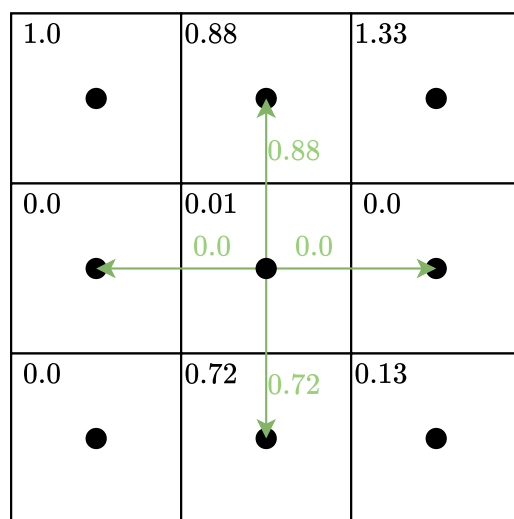


Figura 4: Ejemplo de transiciones de un nodo en el grafo que representa el espacio de búsqueda. El peso asociado a las aristas se corresponde con el valor que posee la casilla destino.

secuencia de POI's que debe sobrevolar el avión espía, escogidos intencionalmente para probar la correcta implementación del sistema.

Junto con la realización de los experimentos correspondientes a los escenarios aportados con el enunciado, el alumno tendrá que realizar una batería de pruebas adicional, con los escenarios y POI's que estime oportunos, para validar el correcto funcionamiento del sistema.

3. Código proporcionado

Se proporciona un código base escrito en Python (lenguaje sobre el que se desarrollará la implementación del sistema) que el que el alumno debe trabajar para implementar el sistema de búsqueda propuesto. El objetivo por parte del alumno consiste en completar el código de ciertas funciones sin implementar que son clave en el sistema. Funcionalidades básicas para la generación de gráficas que permiten visualizar la solución, junto con otras funcionalidades para la generación aleatoria de radares dentro de un área, así como la estructura básica de algunas clases del problema, son proporcionadas por el código base.

Se insta activamente a los alumnos a emplear la biblioteca `Numpy` de Python para las operaciones con *arrays*, que permitirán obtener un rendimiento muy superior al de las listas nativas de Python, además de facilitar la implementación de funciones que operan con matrices y/o vectores. Se trata de una biblioteca muy popular y ampliamente usada que todo programador en Python debería conocer y manejar.

Además de las funciones a completar por parte del alumno, si éste estima oportuno implementar funciones adicionales con finalidades auxiliares, para refactorizar código o para facilitar la comprensión del sistema, le está totalmente permitido hacerlo. Se recuerda que es una buena práctica de programación el comentar todo lo que se pueda (sin caer en el exceso), para facilitar la comprensión del código tanto por parte de los alumnos como por parte del profesor.

Como se ha comentado previamente en el presente documento, junto con el código base se aporta un fichero `JSON` que contiene la configuración de distintos escenarios que deberán ser probados por los alumnos para comprobar el correcto funcionamiento de sus sistemas. Se recuerda que el alumno debe ampliar esta batería de pruebas incluyendo

escenarios que prueben situaciones donde se compruebe la robustez del sistema. Cada uno de estos escenarios posee la siguiente estructura:

```
1 {
2   "scenario_0": {
3     "max_lat": 37.29139325161781, // Identificador del escenario
4     "min_lat": 37.21979775354181, // Latitud máxima en el mapa
5     "max_lon": -115.78524417824534, // Latitud mínima en el mapa
6     "min_lon": -115.8885843284312, // Longitud máxima en el mapa
7     "W": 16, // Longitud mínima en el mapa
8     "H": 16, // Número de puntos horizontales que tendrá la malla
9     "n_radars": 1, // Número de puntos verticales que tendrá la malla
10    "POIs": [[37.21979775, -115.88858433], // Número de radares distribuidos aleatoriamente por el mapa
11             [37.21979775, -115.81969089]] // Puntos de interés a visitar
12  }
13 }
```

De manera que el alumno tendrá que replicar dicha estructura para agregar los escenarios que estime relevantes. Finalmente, es importante destacar que el IDE de desarrollo a utilizar durante la práctica será *Visual Studio Code*, ya que el código base contiene un fichero `launch.json` que permite ejecutar en modo *debug* el sistema, automatizando el paso de argumentos al programa y facilitando la depuración de errores. Adicionalmente, se permitirá el uso de *PyCharm* como IDE de desarrollo alternativo.

4. Evaluación

En la siguiente secuencia de apartados se detallan las distintas puntuaciones asociadas a cada uno de los apartados que el alumno deberá realizar. En total el **bloque práctico** ocupa **7 puntos** de la práctica y la **memoria** a realizar representa los **3 puntos** restantes.

4.1. Modelado del problema

Este apartado suma un total de **2 puntos**. Su desglose es:

- Modelar matemáticamente los estados y operadores del problema (**0.7 pts**).
- Definir el estado inicial y las características que posee un estado final (de aceptación), empleando el modelo matemático desarrollado en el apartado anterior (**0.3 pts**).
- Diseñar y formular **como mínimo dos heurísticas admisibles** para el problema de búsqueda (**1 pt**).

4.2. Implementación del mapa

Este apartado suma un total de **2 puntos**. Su desglose es:

- Implementar la generación del mapa (malla) sobre la que se define el espacio de búsqueda (**1 pt**).
- Implementar la generación del grafo a partir de la malla y las posibles transiciones entre distintas casillas / celdas (**1 pt**).

4.3. Implementación del sistema de búsqueda

Este apartado suma un total de **2 puntos**. Su desglose es:

- Integrar el algoritmo de búsqueda A^* de la biblioteca `networkx` en el sistema, empleando las heurísticas admisibles diseñadas con anterioridad (**2 pts**).

4.4. Experimentación

Este apartado suma un total de **1 punto**. Su desglose es:

- Probar distintos escenarios y recoger sus resultados en la memoria. Se valorará positivamente el uso de gráficas y tablas (**1 pt**).

4.5. Memoria

La memoria de la práctica representa un total de **3 puntos** sobre 10. El objetivo de la memoria es el documentar con detalle la solución que se ha diseñado e implementado, destacando los aspectos técnicos más relevantes. Su desglose es:

- Contenido y explicación del sistema (**1.5 pts**).
- Estructura y presentación del documento (**0.5 pts**).
- Descripción, justificación y aprendizaje del uso de IA generativa (**1 pt**).

Se hace especial hincapié en que el alumno **cuide meticulosamente** la calidad de la memoria, poniendo especial énfasis en las explicaciones claras y formales, así como en el estilo, estética y presentación de la misma. Una buena memoria debería ser capaz de explicar el sistema desarrollado sin necesidad de recurrir prácticamente al código. Esto no significa que haya que agregar código a la memoria, el alumno **deberá evitarlo** en todo momento. Las memorias deberán tener una **portada, índice, introducción, contenido, experimentos y conclusiones** como mínimo, **enumerando** las páginas excepto la correspondiente a la portada. Si el alumno recurre a la literatura científica deberá **citar las fuentes** que haya empleado. Si se agregan referencias, el formato de citación será de libre elección por parte del alumno.

5. Normas

Esta práctica se debe realizar por grupos de 3 alumnos. La entrega de la práctica consistirá en un fichero comprimido ZIP que debe ser entregado por cada grupo a través del enlace proporcionado por Aula Global. El nombre del fichero ZIP debe ser "Práctica IA 2025 <st-code1><st-code2><st-code3>.zip", donde <st-codex>son los seis últimos dígitos del NIA de cada uno de los miembros del grupo.

- Ejemplo: Práctica IA 2025 123456 345678 456789.zip

El fichero ZIP incluirá:

- Carpeta con el código correspondiente, separado en ficheros PY y JSON.
- Memoria técnica de la práctica compilada en formato PDF.
- (Opcional) Fichero de texto (TXT) README con instrucciones e indicaciones necesarias para facilitar la ejecución del sistema y corrección del mismo, si el alumno lo estima oportuno.

Se habilitará un entregador en Aula Global, plataforma a través de la cual los alumnos deberán subir el fichero ZIP descrito anteriormente. La fecha límite para entregar la práctica es el 15 de mayo a las 23:59. No se aceptarán prácticas enviadas por correo electrónico o fuera de la fecha límite de entrega.

6. Uso de IA generativa

Se permite el uso de herramientas de IA generativa, como ChatGPT, siempre que su utilización sea transparente y debidamente documentada. En particular, la IA puede emplearse para:

- Buscar información relevante o ejemplos relacionados con la práctica.
- Obtener explicaciones sobre métodos, bibliotecas o conceptos específicos.
- Generar ideas o posibles enfoques para la solución del problema, siempre que el alumno los analice críticamente y los adapte a su propio desarrollo.

Restricciones

- No está permitido utilizar IA generativa para generar código, algoritmos o fragmentos de texto que sean incorporados directamente en el sistema o en la memoria sin modificaciones ni comprensión por parte del alumno.
- Se espera que el alumno diseñe, implemente y documente la solución de manera razonada y justificada.

Declaración de Uso

El alumno deberá incluir un apartado adicional al final de la memoria de la práctica, donde describirá de forma transparente el uso que ha hecho de la IA generativa. En este apartado deberá especificar:

- Para qué aspectos de la práctica ha empleado la IA.
- En qué medida le ha facilitado la realización de la práctica.
- Cómo ha verificado y adaptado la información obtenida.

Esta declaración será evaluada positivamente en la calificación final de la práctica, en función de la calidad y profundidad del análisis realizado.